

No Lost Media Launcher

Guia Técnico: Migração para Repositório Privado & Impacto no Auto-Update

Este documento analisa os impactos de tornar o repositório **SarmientoCaio/No-Lost-Media-Launcher** privado no GitHub. Privar o repositório protege a propriedade intelectual do código-fonte e da arquitetura do catálogo, mas **interrompe imediatamente o sistema de auto-update integrado** caso não seja adotada uma das arquiteturas de desacoplamento descritas neste relatório.

1. Diagnóstico: O Auto-Update será Afetado?

SIM. O auto-update deixará de funcionar de forma imediata e transparente para os usuários. A infraestrutura atual do Launcher consome dois pontos de acesso no GitHub, ambos bloqueados pelo modo privado:

- **Bloqueio na API de Releases (updateService.ts):** A rota `api.github.com/repos/.../releases/latest`, consultada pelo frontend para exibir o modal de nova versão, retorna **HTTP 404 Not Found** para chamadas não autenticadas em repositórios privados. O código trata 404 como ausência de atualizações, de modo que o usuário nunca saberá que existe uma versão nova.
- **Bloqueio no Endpoint Estático do Tauri (tauri.conf.json):** O plugin `tauri-plugin-updater` busca o arquivo `.../releases/latest/download/latest.json`. No GitHub privado, downloads anônimos de assets de release são bloqueados (retornam 404 ou redirecionamento HTML para a tela de login do GitHub). A validação de assinatura `Ed25519` falha.
- **Bloqueio no Download do Executável (.exe):** Mesmo que o link do instalador seja conhecido, o download do binário `No-Lost-Media-Launcher-Setup.exe` na CDN do GitHub exige cabeçalhos com token de acesso ao repositório.

Ponto de Consulta	Repositório Público (Atual)	Repositório Privado (Sem Ajuste)
GitHub API (/releases/latest)	■ HTTP 200 (JSON público)	■ HTTP 404 Not Found (Bloqueado)
Manifesto Tauri (latest.json)	■ HTTP 302/200 (Acesso livre)	■ HTTP 404 / Redirect Login HTML
Download do Instalador (.exe)	■ Download direto via CDN	■ Acesso negado sem token
Comportamento no App do Usuário	■ Atualização silenciosa em background	■ Nunca encontra atualizações

2. Risco Crítico: Por que NUNCA embutir um Token no App

A solução intuitiva mais rápida para tentar viabilizar o download seria gerar um *Personal Access Token (PAT)* do GitHub e colocá-lo no código do Launcher para autenticar as requisições. **Isso é uma grave falha de segurança:**

RISCO DE VAZAMENTO INTEGRAL DO REPOSITÓRIO PRIVADO

Qualquer executável distribuído aos usuários pode ser inspecionado. Com ferramentas simples (Strings, Proxies HTTP como Fiddler ou Wireshark, ou descompiladores JS), o token pode ser extraído em minutos. Um usuário mal-intencionado ganha acesso total ao seu código e issues.

- **Descompilação Trivial de Frontend:** O código do launcher empacotado no executável do Tauri contém os arquivos JavaScript do frontend. Mesmo minificados, tokens do GitHub (com prefixo `ghp_`) são facilmente localizáveis.
- **Interceptação de Tráfego de Rede:** Qualquer usuário monitorando sua própria placa de rede verá o cabeçalho `Authorization: Bearer [token]` enviado para a API do GitHub nas requisições do launcher.
- **Princípio Zero-Trust:** O cliente desktop é considerado ambiente inseguro. Credenciais com permissões de acesso a repositórios privados **jamaiz** devem ser embutidas em binários entregues ao público.

3. Outras Preocupações Operacionais com Repo Privado

Além da quebra do autoupdate, a mudança para repositório privado acarreta mudanças importantes no dia a dia do projeto:

■ Cota de Minutos do GitHub Actions (CI/CD)

Repositórios públicos possuem minutos de execução **ilimitados e gratuitos** no GitHub Actions. Repositórios privados no plano Free possuem cota de **2.000 minutos/mês** compartilhada em toda a conta. Compilações de Rust e MSVC levam de 3 a 5 minutos por execução.

■ Quebra de Links Externos (Site e Redes Sociais)

Qualquer link no site `nolost.media`, em vídeos, posts ou tutoriais que aponte para `github.com/SarmientoCaio/No-Lost-Media-Launcher` passará a retornar erro 404 para quem não fizer parte da organização.

■ Perda de Issues e Feedback da Comunidade

Usuários externos não poderão mais abrir Issues no GitHub relatando bugs em jogos ou melhorias no launcher. Será indispensável disponibilizar um canal alternativo (ex: Discord oficial, formulário ou e-mail de suporte).

■ Armazenamento e Quotas de Releases

Releases em repositórios privados contam diretamente no limite de armazenamento da conta GitHub (1 GB no plano gratuito), enquanto em repositórios públicos os instaladores não reduzem essa cota de storage.

4. As 3 Soluções Arquiteturais Recomendadas

Para manter o código-fonte 100% privado e garantir que o auto-update continue funcionando de forma anônima e gratuita, existem três soluções adotadas na indústria de software desktop:

■ Solução 1: Repositório Público Dedicado a Releases (Altamente Recomendada)

Esta é a arquitetura padrão utilizada por softwares fechados (ex: Obsidian, Notion). Separa-se o repositório de código do repositório de distribuição:

- **Repositório de Código (Privado):** SarmentoCaio/No-Lost-Media-Launcher (guarda o código-fonte, histórico e lógica).
- **Repositório de Releases (Público):** SarmentoCaio/nlm-releases (contém **somente** tags, instaladores .exe e o latest.json).
- **Funcionamento:** O build roda no repositório privado. Ao concluir, publica o release diretamente no repositório público usando o GitHub CLI (gh release create --repo ...). O launcher consulta o repo público.
- **Vantagens:** Custo zero de infraestrutura; aproveita a CDN global ultrarrápida do GitHub; sem chaves no client; código 100% protegido.

■ Solução 2: Cloudflare R2 / AWS S3 com Domínio Próprio (A mais Profissional)

Armazenar o latest.json e os instaladores em um bucket Cloudflare R2 conectado a um subdomínio oficial (ex: https://updates.nolost.media/latest.json):

- **Cloudflare R2:** Compatível com S3 e com **custo zero de tráfego de saída (zero egress fee)**. Milhares de downloads de 7 MB não geram custos de banda.
- **Vantagens:** Independência total de plataformas; URL com a marca própria No Lost Media; imune a eventuais bloqueios ou políticas do GitHub.

■ Solução 3: Cloudflare Worker como Gateway Reverso Autenticado

Manter as releases no próprio repositório privado e usar um Cloudflare Worker no meio do caminho para autenticar com segurança:

- O Launcher consulta https://updates.nolost.media/latest.json sem token.
- O Worker (serverless na nuvem) intercepta a requisição, anexa o GitHub Token armazenado com segurança em seus secrets de backend, busca o release no repositório privado e entrega o conteúdo ao launcher. O token nunca sai do servidor seguro.

5. Comparativo das Soluções

Critério	Solução 1: Repo Público Releases	Solução 2: Cloudflare R2 (S3)	Solução 3: Cloudflare Worker
Privacidade do Código	■■■■■ 100% Protegido	■■■■■ 100% Protegido	■■■■■ 100% Protegido
Custo Financeiro	■■■■■ Gratuito	■■■■■ Gratuito (Zero Egress)	■■■■■ Gratuito (Plano Free)
Facilidade de Setup	■■■■■ Simples (5 min)	■■■■ Médio (Bucket + DNS)	■■■ Avançado (Código Worker)
Independência do GitHub	■■■ Vinculado ao GitHub	■■■■■ 100% Independente	■■■ Depende da API GitHub
Velocidade e CDN	■■■■■ CDN GitHub Fastly	■■■■■ CDN Cloudflare Anycast	■■■■ Proxy intermediário

6. Plano de Ação Recomendado (Passo a Passo)

A **Solução 1 (Repositório Público de Releases)** é a rota de menor atrito, custo zero e máxima estabilidade. Abaixo está o roteiro de migração para que nenhum usuário fique sem atualização:

1■■ Passo 1: Criar o Repositório de Releases no GitHub

Crie um repositório **público** vazio no GitHub chamado SarmentoCaio/No-Lost-Media-Releases (ou nlm-releases). Não adicione arquivos de código nele, apenas uma descrição simples informando que o repositório armazena os instaladores e atualizações oficiais do No Lost Media Launcher.

2■■ Passo 2: Atualizar os Endpoints de Update no Projeto

No repositório do projeto, atualize os dois arquivos responsáveis pelas consultas de versão:

- Em `apps/launcher/src-tauri/tauri.conf.json`:
`"endpoints": ["https://github.com/SarmentoCaio/No-Lost-Media-Releases/releases/latest/download/latest.json"]`
- Em `apps/launcher/src/services/updateService.ts`:
`export const GITHUB_REPO = "SarmentoCaio/No-Lost-Media-Releases";`

3■■ Passo 3: Publicar a Versão de Transição no Repo de Releases

Faça o build da nova versão e publique no repositório de releases usando o GitHub CLI:

```
gh release create v0.1.4 release_assets/* --title "No Lost Media Launcher 0.1.4" --repo SarmentoCaio/No-Lost-Media-Releases
```

Espelhe também os arquivos no repo antigo caso queira que os usuários antigos encontrem a versão 0.1.4.

4■■ Passo 4: Alterar a Visibilidade do Repositório de Código para Privado

No GitHub, acesse *Settings* → *Danger Zone* → *Change repository visibility* no repositório principal e selecione **Private**. A partir desse momento, seu código-fonte, commits e histórico estão completamente protegidos e invisíveis para o público externo.

ESTRATÉGIA DE TRANSIÇÃO SUAVE (IMPORTANTE)

Para que os usuários que já utilizam a versão 0.1.3 atualizem automaticamente sem intervenção, publique a versão 0.1.4 em ambos os repositórios (antigo e novo) antes de privar o repositório principal. Assim que todos tiverem recebido a 0.1.4, pode-se privar o repositório antigo com total segurança.